

Komparativna analiza sinhronih komunikacionih paradigmi u IoT mikroservisnim sistemima

1. Uvod i Arhitektura Sistema

Cilj ovog projekta je istraživanje i evaluacija performansi tri dominantna sinhrona modela komunikacije: REST (HTTP/1.1 sa JSON-om), gRPC (HTTP/2 sa binarnim Protobuf-om) i GraphQL (HTTP sa fleksibilnim klijentskim upitima) u specifičnom kontekstu Interneta stvari (IoT).

Sistem simulira ekosistem za praćenje kvaliteta vazduha (Environmental Monitoring). IoT senzori (Edge sloj) generišu i šalju vremenski serijalizovane podatke, dok mikroservisi (Cloud/Server sloj) obrađuju upise i omogućavaju klijentima selektivno praćenje i naprednu analitiku.

- Tehnološki stek:
- Baza podataka: PostgreSQL 16 (optimizovana za vremenske serije).
- REST Servis: ASP.NET Core 10.0 (C#) sa Dapper ORM-om i ugrađenom OpenAPI (Swagger) dokumentacijom.
- gRPC Servis: ASP.NET Core 10.0 (C#) sa nativnom HTTP/2 podrškom i Protobuf serijalizacijom.
- GraphQL Servis: Node.js (v22) sa Apollo Server platformom i pg drajverom za direktan pristup bazi.
- Orkestracija: Docker Compose (kompletna kontejnerizacija svih servisa u izolovanoj mreži).

2. Dizajn i Optimizacija Baze Podataka

Za potrebe skladištenja IoT podataka kreirana je tabela `sensor_data` unutar PostgreSQL baze podataka. Tabela je optimizovana za upite visokog intenziteta i pretragu po vremenskom opsegu.

Kompozitni indeks nad kolonama `device_id` i `recorded_at` omogućava bazi podataka da u konstantnom vremenu $O(\log N)$ locira specifični uređaj i izvrši agregaciju ili vrati najnoviji podatak bez skeniranja cele tabele.

3. Implementacija Mikroservisa

3.1 REST Servis (ASP.NET Core)

REST servis koristi standardni arhitektonski stil, gde se podaci razmenjuju u JSON formatu preko HTTP/1.1 protokola.

3.2 gRPC Servis (.NET & Protobuf)

gRPC servis implementira binarnu komunikaciju visokih performansi preko HTTP/2 protokola. Ugovor se definiše pomoću Protocol Buffers kompajlera.

3.3 GraphQL Servis (Node.js & Apollo Server)

GraphQL pruža fleksibilnost klijentima, dozvoljavajući specifikaciju tačnih polja koja su im potrebna, čime se rešava problem mrežnog rasipanja podataka (over-fetching).

3.4 Orkestracija (Docker Compose)

Sve komponente su spakovane u izolovanu Docker mrežu.

4. Merenje i evaluacija performansi

U ovom poglavlju predstavljeni su rezultati testiranja implementiranih mikroservisa (REST, gRPC, GraphQL) pod različitim opterećenjima i specifičnim IoT scenarijima. Testiranje je sprovedeno unutar izolovanog Docker okruženja radi konzistentnosti merenja.

4.1 Evaluacija latencije i propusnosti (k6 Load Test)

Za generisanje opterećenja korišćen je alat k6. Merenja su vršena za 10, 100 i 500 virtuelnih korisnika (VUs).

Tabela 1: Performanse protokola - Scenario A (High-Frequency Ingestion)

Protokol	Broj korisnika (VUs)	RPS (Req/s)	Prosečna latencija (ms)	p95 Latencija (ms)
REST	10	9.9	5.88	6.22
REST	100	99.3	5.91	12.93
REST	500	410	3.03	4.2
GraphQL	10	9.9	12.21	30.14
GraphQL	100	98.6	10.72	43.15
GraphQL	500	409.7	4	7.09
gRPC	10	9.9	4.73	8.87
gRPC	100	98.9	4.67	8.04
gRPC	500	408.6	2.83	4.38

Analiza (Scenario A): Sva tri protokola odlično skaliraju do 410 RPS. Međutim, gRPC ostvaruje najnižu latenciju (2.83ms) zahvaljujući efikasnom binarnom Protobuf formatu. REST nudi stabilne performanse (3.03ms), dok GraphQL beleži najvišu latenciju usled procesorskog "overhead"-a pri parsiranju upita.

Tabela 2: Performanse protokola - Scenario B (Selective monitoring)

Protokol	Broj korisnika (VUs)	RPS (Req/s)	Prosečna latencija (ms)	p95 Latencija (ms)
REST	10	9.8	18.78	18.06
REST	100	97.7	18.55	23.67
REST	500	405.9	13.9	19.71
GraphQL	10	9.8	19.22	22.73
GraphQL	100	97.7	19.04	27.27
GraphQL	500	405.5	13.62	18.68
gRPC	10	9.8	20.49	29.28
gRPC	100	97.3	18.12	24.74
gRPC	500	400	23.03	47.92

Analiza (Scenario B): U read-heavy scenariju selektivnog praćenja, GraphQL dominira sa najnižom latencijom (13.62ms) jer hirurški precizno rešava problem "over-fetching"-a. REST ostaje stabilan, dok gRPC beleži pad performansi usled krutosti strukture poruka pri povlačenju kompletnih entiteta.

Tabela 3: Performanse protokola - Scenario C (Heavy Querying)

Protokol	Broj korisnika (VUs)	RPS (Req/s)	Prosečna latencija (ms)	p95 Latencija (ms)
REST	10	9.7	29.96	41.78
REST	100	95.2	32.7	48.38
REST	500	314.8	302.94	589.25
GraphQL	10	9.9	11.71	21.62
GraphQL	100	97.4	20.79	42.19
GraphQL	500	400	27.91	61.21
gRPC	10	9.7	24.81	32.74
gRPC	100	94.7	31.45	40.22
gRPC	500	253.9	604.82	1320

Analiza (Scenario C): Kod složenih agregacija, GraphQL je pokazao najveću efikasnost (27.91ms) zahvaljujući asinhronom radu resolvera. REST usporava usled zasićenja thread pool-a (302ms), a gRPC doživljava drastičan pad performansi (604ms) usled zagušenja deljene multipleksirane HTTP/2 konekcije sporim upitima iz baze.

4.2 Analiza mrežnog saobraćaja (Veličina odgovora)

Merenje je izvršeno praćenjem veličine odgovora u bajtovima radi provere efikasnosti propusnog opsega.

Tabela 4: Poređenje veličine mrežnog paketa (Payload Size)

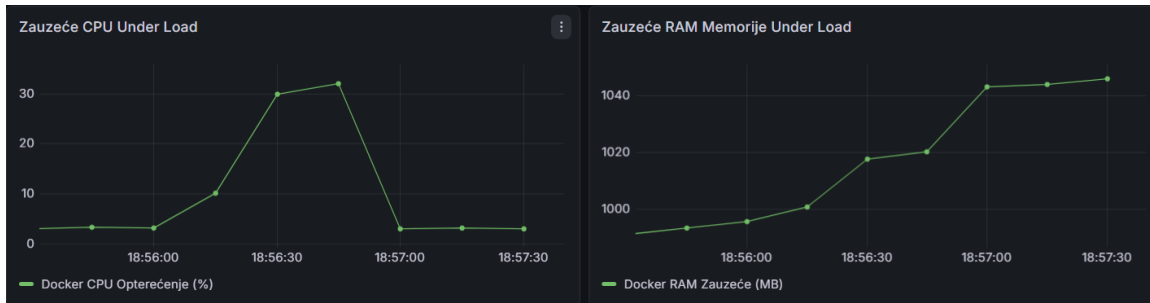
Protokol	Scenario A (B)	Scenario B (B)	Scenario C (B)
REST	175	362	293
gRPC	163	248	224
GraphQL	259	323	397

Analiza veličine paketa: gRPC isporučuje najoptimalniju kompresiju zahvaljujući binarnom Protobuf formatu na svim nivoima. Sa druge strane, u selektivnom Scenario B, GraphQL efikasno smanjuje prenos nepotrebnih polja karakterističnih za JSON.

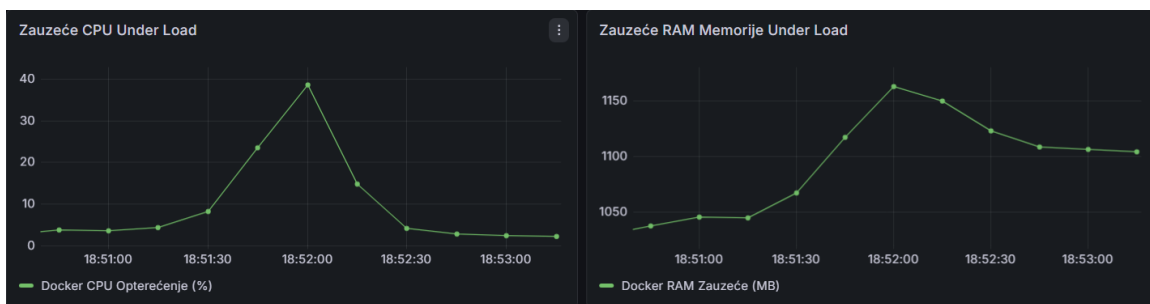
4.3 Analiza potrošnje resursa (CPU i RAM) pod opterećenjem

Merenje potrošnje resursa integracijom alata cAdvisor, Prometheus i Grafana vizuelizovano je na osnovu Scenarija A.

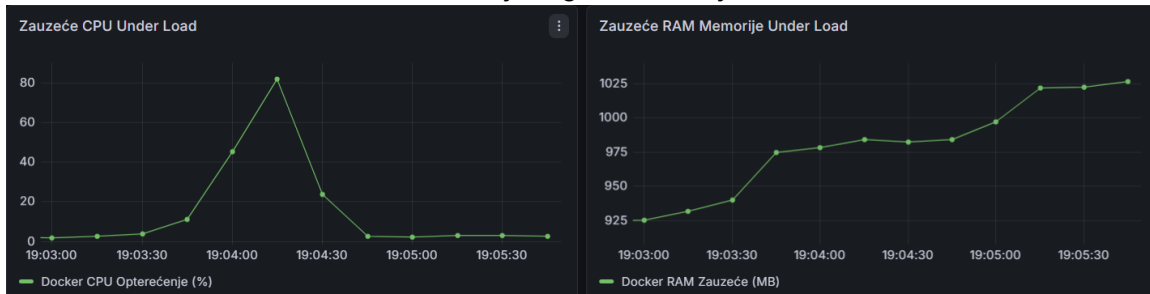
- Slika 1: Zauzeće CPU i RAM memorije – REST kontejner.



- Slika 2: Zauzeće CPU i RAM memorije – GraphQL kontejner.



- Slika 3: Zauzeće CPU i RAM memorije – gRPC kontejner.



Analiza hardverskih resursa: REST pokazuje stabilno zauzeće CPU-a (32%) uz umeren rast RAM-a. GraphQL beleži najveće memorijsko opterećenje usled generisanja i obrade apstraktnog sintaksnog stabla (AST). Sa druge strane, gRPC ostvaruje najviše opterećenje procesora (preko 80%) zbog brzog protoka zahteva, ali održava ubedljivo najniži i najstabilniji memorijski otisak usled izostanka dugačkog JSON parsiranja.